Modeling Human Activity States Using Hidden Markov Models

Samuel Kwizera Ihimbazwe

Repository: github.com/Samkwizera/hidden-markov-models

1. Background and Motivation

I built this for my grandmother. She lives in the Western Province of Rwanda, far from me, and she is hard to reach, so my family often goes days without knowing how she is or whether she is moving around normally. What I want is a wearable monitor that runs on a smartphone at her waist and lets us follow her movements from a distance, so that if we see something unusual, such as a sudden hard movement or a long time without any movement, we can call someone nearby or go to her. The phone labels each moment as standing, walking, vigorous movement or still. Jumping is my stand-in for sudden high-impact movement such as a stumble. Still is the state that would worry us most, because it means either she has stopped moving or the phone has been put down and we can no longer see how she is. The phone never observes her activity directly. It only sees noisy accelerometer and gyroscope readings, and one activity tends to last a while before the next begins. A Hidden Markov Model handles both: its emission model describes what each activity looks like in the sensors, and its transition model describes how one activity follows another.

2. Data Collection and Pre-processing

I recorded all the data with Sensor Logger (v1.68) on my iPhone X (iOS 16.7), holding the phone at waist level. The app logged the accelerometer (gravity removed, m/s^2) and the gyroscope (rad/s). Each training clip is 5 to 10 s of a single activity, and I recorded all of them on 26 Oct 2026.

Participant	Phone	Sampling rate
Samuel Kwizera Ihimbazwe	iPhone X (iOS 16.7)	100 Hz (10 ms interval)

Table 1. Recording device and sampling rate.

Activity	Training files	Total duration	Mean clip length
Standing	14	136.9 s	9.8 s
Walking	14	115.9 s	8.3 s
Jumping	14	116.7 s	8.3 s
Still (phone on table)	14	116.3 s	8.3 s

Table 2. Training data per activity after cleaning. Every activity is above the 1 min 30 s minimum. Four more files were kept aside for testing.

Each of the videos in Table 3 shows my full body and face while the phone records the activity.

Activity	Video link
Standing	https://drive.google.com/file/d/1YgA6UkHaGOPdqqx5GI96xit061Aif5J6/view?usp=sharing
Walking	https://drive.google.com/file/d/1x27T9-VPxknB-DW4e_CIrS3akjHLhK4d/view?usp=sharing
Jumping	https://drive.google.com/file/d/1fWQBKvB6WefJN4QSezeShmdJOlVRR3uY/view?usp=sharing
Still	https://drive.google.com/file/d/1DGSEliRQ-BV2BMIkxrYEzLXstcKhLoSF/view?usp=sharing

Table 3. Data collection videos.

To clean the data, prepare_data.py does three things to each recording:

1. It merges the accelerometer and gyroscope files on their shared nanosecond timestamp and drops duplicate timestamps.
2. It cuts 0.5 s from each end, where my finger tapped the screen to start and stop logging.
3. It saves the result as data/train/<activity>/<activity>_NN.csv, with a timestamp column and a label column.

iOS timestamps drift a little around the nominal 10 ms, so the real sampling rate varies slightly within a file. I interpolate every recording linearly onto an exact 100 Hz grid. After that, every window has the same number of samples and the same FFT frequency bins. A phone that records at another rate, such as 50 Hz, would go onto the

same grid, so its features would be comparable with mine. Human movement has almost nothing above 5 Hz, so 100 Hz is far more than the model needs.

I cut the signals into 2 s windows (200 samples at 100 Hz) with a 1 s hop, which gives 50 % overlap. Walking (about 1.7 Hz) and jumping (about 2 Hz) are periodic, and a 2 s window holds 3 to 4 full cycles. That is enough for a stable variance estimate and a clear FFT peak. The FFT resolution is fs/N = 100/200 = 0.5 Hz, fine enough to separate the step rhythm from the near-zero content of standing and still. A longer window would blur the transitions the HMM is supposed to find. Windowing produces 399 training windows: 114 standing, 94 walking, 95 jumping and 96 still.

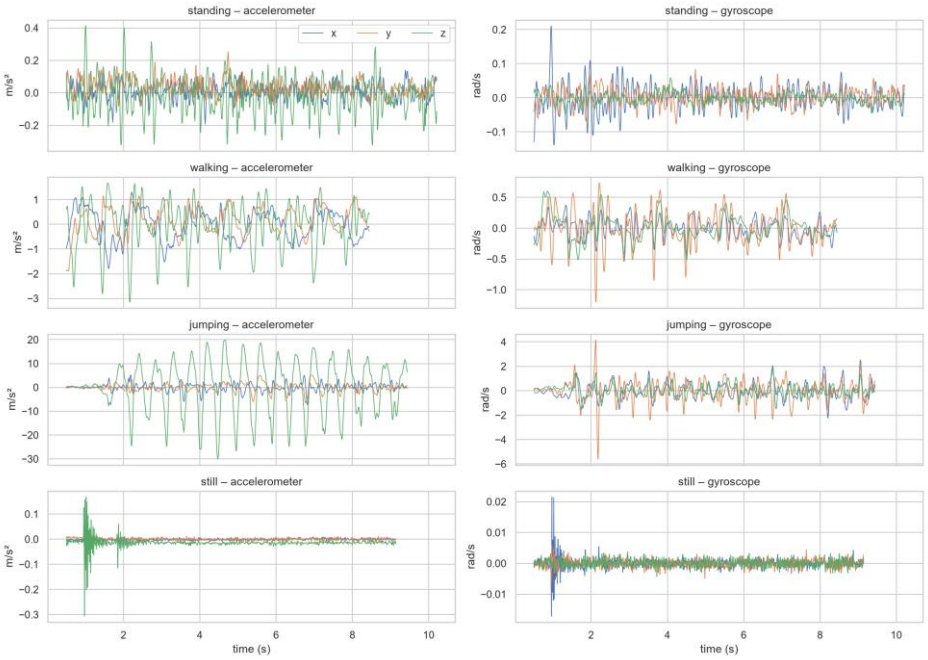


Figure 1. Raw accelerometer (left) and gyroscope (right) signals for one training clip of each activity.

3. HMM Setup and Implementation Details

3.1 Feature extraction

Each window becomes one 10-dimensional observation. Every feature is computed on the magnitude of acceleration |a| or angular velocity |ω|, so the way the phone is turned in my hand does not change it.

Feature	Domain	Why it separates the activities
acc mean, RMS, SMA	Time	Overall movement intensity: still < standing < walking < jumping
acc std	Time	Jumping produces large impact spikes; standing only small body sway
gyro mean, std	Time	Walking rotates the hips; still has no rotation at all
acc corr (x, z)	Time	Axes move together during a periodic step; noise is uncorrelated
acc dominant freq.	Frequency (FFT)	Step or jump rate (about 1.7 to 2 Hz); random for standing and still
acc spectral energy	Frequency (FFT)	Total non-DC power of the window (Parseval)
gyro dominant freq.	Frequency (FFT)	Rhythm of hip rotation during gait

Table 4. The 10 features (7 time-domain, 3 frequency-domain).

The intensity features range from about 0.003 m/s² when the phone is still to about 10 m/s² when I jump, so I take log10(x + 0.001) of them. On the log scale each state's distribution is much closer to the Gaussian shape the emission model assumes. I then Z-score every feature with the mean and standard deviation of the training windows, and apply those same values to the test set so no test information leaks into training. I chose Z-score over min-max scaling because under min-max a few extreme jumping spikes would squeeze every other state into a tiny range. Z-scores are also in standard-deviation units, which is how a Gaussian emission is written. Figure 2 shows the four states are well separated on most features.

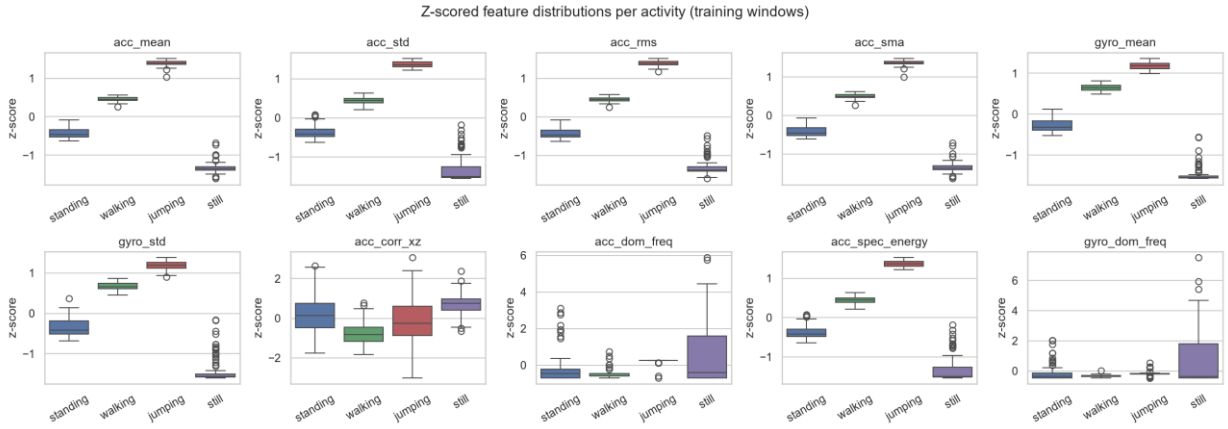


Figure 2. Z-scored feature distributions per activity (training windows).

3.2 Model components

Element	Definition in this project
Hidden states Z	standing, walking, jumping, still
Observations X	10-dimensional normalised feature vector per 2 s window
Transitions A	4 × 4 matrix P(z_t = j | z_t−1 = i); one step = 1 s (the window hop)
Emissions B	Diagonal-covariance Gaussian N(x; μ_k, σ²_k) for each state
Initial π	Probability of each activity at the start of a sequence

Table 5. HMM components.

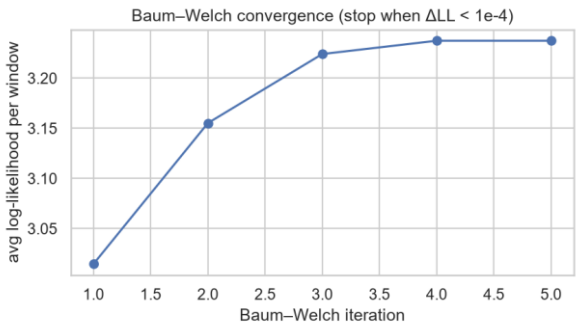
3.3 Algorithms and implementation

I implemented the HMM from scratch in numpy (hmm.py, class GaussianHMM). I decode with the Viterbi algorithm, which returns the single most likely activity sequence. My metrics (accuracy, sensitivity and specificity) are computed per window from one hard label, so a single best path is what they need. Viterbi's transition term also overrides isolated one-window errors, which cuts false positives and raises specificity.

I train with Baum-Welch, which is EM with a scaled forward-backward pass. It re-estimates π, A, μ and σ² to maximise the likelihood of the training sequences, and it stops once the average log-likelihood per window changes by less than $\varepsilon = 10^{-4}$. It converged after 5 iterations (Figure 3). EM only finds a local optimum and can swap which state means which activity, so I start it from the labelled data: per-class means and variances, plus transition counts. A Dirichlet pseudo-count of 1 keeps every transition possible, and a variance floor stops any state's variance from shrinking to zero.

A single training clip contains only one activity and therefore no transitions. To give the model transitions to learn from, I chained the 56 clips in random order (seed 42) into 6 training sequences that imitate someone switching between activities.

4. Results and Interpretation



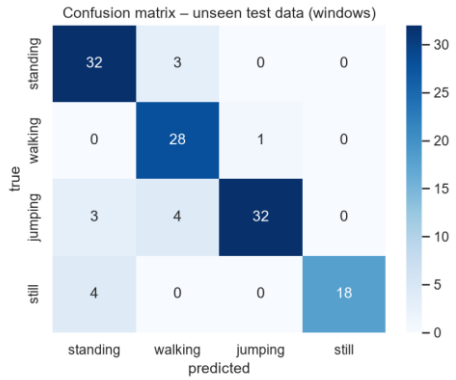


Figure 3. Baum-Welch convergence (left). Figure 4. Confusion matrix on the unseen test data (right).

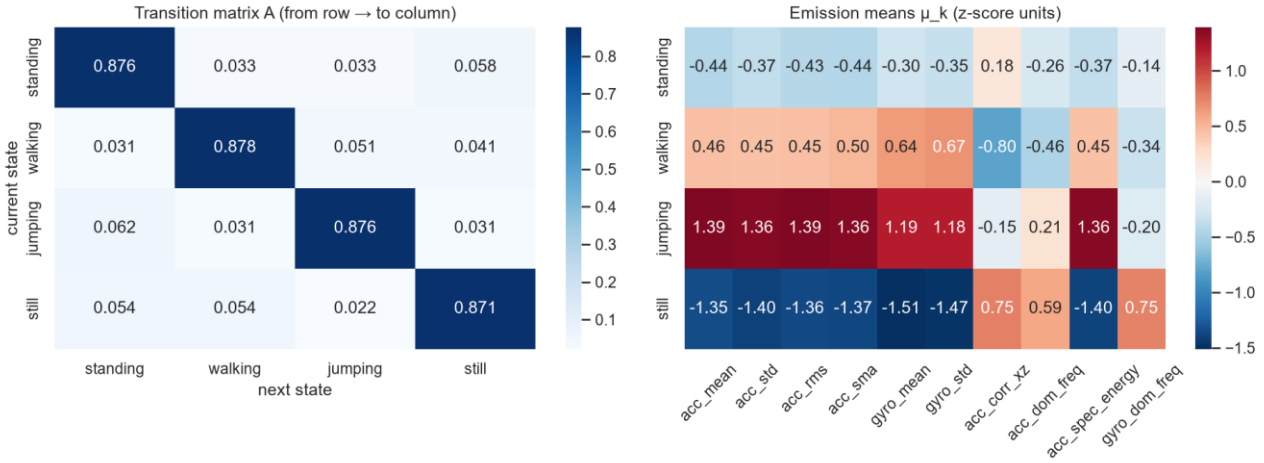


Figure 5. Learned transition matrix A (left) and emission means μ_k in z-score units (right).

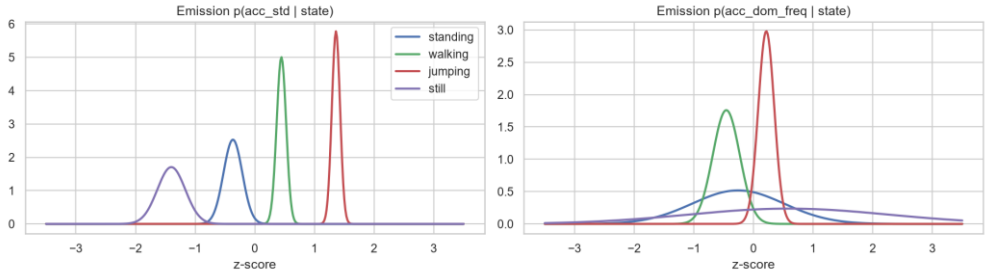


Figure 6. Learned Gaussian emission densities for acc std and acc dominant frequency.

The diagonal of A is about 0.88, which means the model expects to stay in a state for 1/(1 − 0.88) ≈ 8 s, about the length of my clips. The emission means in Figure 5 rank the states by intensity: still sits about 1.4 standard deviations below average on every intensity feature, and jumping about 1.4 above. Still also has a very wide variance on the dominant-frequency features (Figure 6), because when nothing moves the FFT peak lands on random noise.

I tested the model on 4 files it never saw during training or normalisation. Three are continuous recordings of about 40 s each, made in separate sessions after the training clips, in which I did standing → walking → jumping → still back to back. Unlike the training clips, they contain real transitions between activities. I took the ground-truth boundaries from these recordings. The fourth file is a standing recording I made on 23 Oct 2026, three days before the training session.

State (Activity)	Number of Samples	Sensitivity	Specificity	Overall Accuracy
Standing	35	0.914	0.922	0.880
Walking	29	0.966	0.927	0.880
Jumping	39	0.821	0.988	0.880
Still	22	0.818	1.000	0.880

Table 6. Evaluation on unseen data (samples are 2 s windows; Viterbi decoding with the Baum-Welch model).

Accuracy was 0.892, 0.857 and 0.872 on the three continuous sessions and 1.000 on the standing file from the other day. In every session the model finds all four activities in the right order (Figure 7), and every error falls within about 3 s of an activity boundary. The model built from the labelled estimates alone, before Baum-Welch, scored 0.912 on the same data.

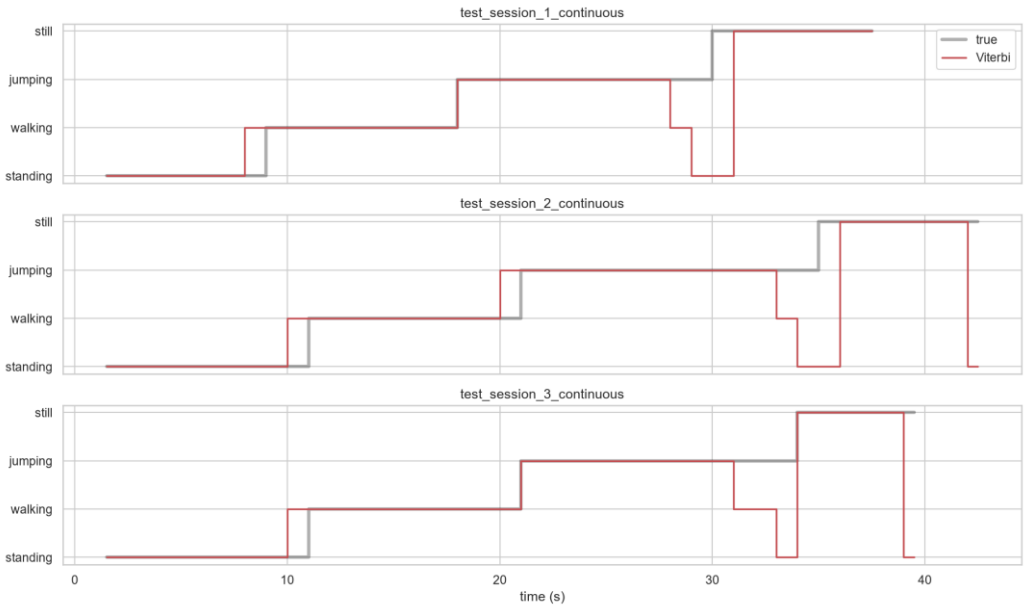


Figure 7. True (grey) vs Viterbi-decoded (red) activity sequence for the three continuous test sessions.

5. Discussion and Conclusion

Walking (sensitivity 0.97) and still (specificity 1.00) were the easiest activities to recognise. Still is the only state with almost no signal, and walking has a clear rhythm of about 1.7 Hz with strong hip rotation. Jumping and still had the lowest sensitivity (0.82), and Figure 7 shows that most of those errors come from my labels. At the end of every jumping bout the model decodes jumping → walking → standing → still. That is what I actually did: I slowed down, walked to the table and put the phone down, but my labels called that whole stretch jumping. The still windows predicted as standing are the moments when I was placing the phone down or picking it up at the end of a recording.

The transition matrix strongly favours staying in the same state (about 0.88). That is realistic, since activities last many seconds, and it is what lets Viterbi ignore a single noisy window. The off-diagonal values are almost uniform, though, because I built the training sequences by chaining clips in random order. In daily life some transitions are much rarer than others: going from still to jumping means picking the phone up first. Real activity bouts also last minutes, so the self-transition probabilities should be nearer 0.98. For my grandmother's monitor, A should be learned from long recordings of her ordinary days.

Even standing still, my body sway produces about 0.1 m/s² of signal, and that small amount of noise is exactly what separates standing from still. At 100 Hz the Nyquist limit is 50 Hz, far above any frequency in human movement, so the sampling rate never held the model back. 25 to 50 Hz would probably work as well and save battery on my grandmother's phone. Baum-Welch raised the training likelihood but lowered test accuracy from 0.912 to 0.880. EM maximises likelihood, and a higher likelihood does not guarantee better classification: here it widened the state variances to cover the mixed windows at activity boundaries.

Several changes would improve the model. It needs more data: all of it is from me, and it should include other people (especially my grandmother and others her age), other phones and other placements such as a pocket or bag. Long natural recordings with real transitions would let A reflect actual behaviour, and labels synchronised with video would fix the boundary errors. Full-covariance Gaussians or Gaussian mixtures could model the emissions better, and new features such as jerk or step-band energy ratio could help. A barometer could detect a drop in height during a fall, and a semi-Markov model could represent how long each activity lasts.

A Gaussian HMM built from scratch with 7 time-domain and 3 FFT features, trained with Baum-Welch and decoded with Viterbi, recognised standing, walking, jumping and still on unseen recordings with 88 % accuracy (91 % with the labelled estimates). Almost all of its errors fall at activity transitions. For my grandmother, this suggests a phone at her waist could show my family when she is moving normally and warn us when something looks unusual, so we can get help to her, although so far I have only tested it on my own data.